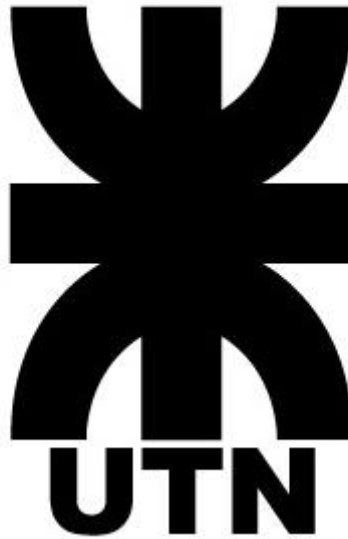


UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

Programación 2



Food Store

Sistema de Gestión de Pedidos de Comida

Tipo de entrega	Documentación Académica y Técnica
Materia	Programación 2
Docente	Ariel Enferrel
Integrantes	Castro Exequiel Cajales Benjamin Piriz Agustina Senatore Genaro
Comisión / Aula	Comision 2
Fecha de entrega	22/06/2026
Repositorio / Video	https://github.com/genarosenatore/TPI-Prog-2-Castro-Exequiel-Cajales-Benjamin-Piriz-Agustina-Senatore-Genaro.git

Índice

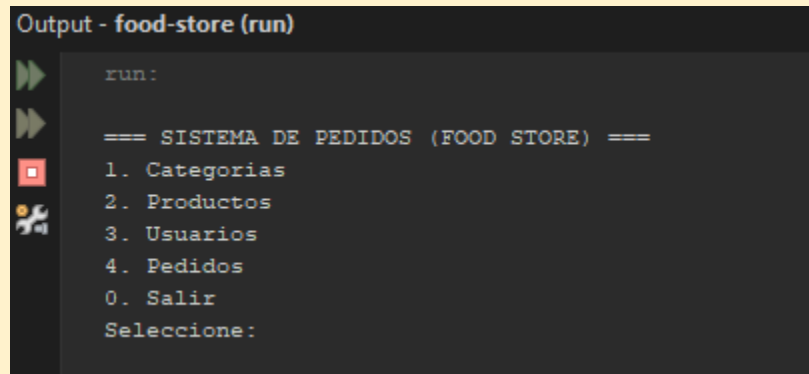
Sección	Página
1. Introducción	3
2. Marco teórico	3
3. Tecnologías utilizadas	4
4. Modelo y entidades	4
5. Arquitectura del sistema	5-7
6. Persistencia y base de datos	8
7. Funcionalidades implementadas	8-9
8. Decisiones técnicas	9
9. Dificultades y soluciones	9
10. Capturas del sistema funcionando	10-14
11. Pruebas realizadas	14
12. Bibliografía / Webgrafía	14
13. Links finales	14

1. Introducción

El presente documento describe el desarrollo del Trabajo Práctico Integrador de la materia Programación 2. El proyecto se denomina Food Store y consiste en una aplicación de consola desarrollada en Java para la gestión de pedidos de comida.

El sistema permite administrar categorías, productos, usuarios y pedidos, utilizando Programación Orientada a Objetos, persistencia mediante JDBC y una base de datos relacional MySQL. La interacción se realiza a través de un menú por consola, sin frontend web ni login, de acuerdo con el alcance definido para la asignatura.

MENÚ PRINCIPAL FUNCIONANDO



```
Output - food-store (run)

run:

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
```

2. Marco teórico

2.1 Programación Orientada a Objetos

La Programación Orientada a Objetos permite modelar el problema mediante clases, objetos, atributos y métodos. En el proyecto se aplican conceptos como encapsulamiento, constructores, herencia, sobrescritura de métodos y relaciones entre entidades.

2.2 JDBC

JDBC es la tecnología utilizada para conectar una aplicación Java con una base de datos relacional. En este sistema se utiliza el driver MySQL Connector/J, consultas SQL parametrizadas con PreparedStatement y lectura de resultados mediante ResultSet.

2.3 Base de datos relacional

La información del sistema se almacena en una base de datos MySQL. Las tablas principales son categoría, producto, usuario, pedido y detalle_pedido. Las relaciones entre tablas permiten representar asociaciones como productos asociados a categorías, pedidos asociados a usuarios y detalles asociados a pedidos.

2.4 Patrón DAO y separación por capas

El patrón DAO separa el acceso a datos de la lógica de negocio. En el proyecto, las clases DAO contienen las consultas SQL, las clases Service contienen validaciones y reglas, y la capa UI contiene únicamente la interacción con el usuario.

2.5 Baja lógica

La baja lógica consiste en marcar un registro como eliminado sin borrarlo físicamente de la base de datos. Para esto se utiliza el campo eliminado en las tablas principales.

3. Tecnologías utilizadas

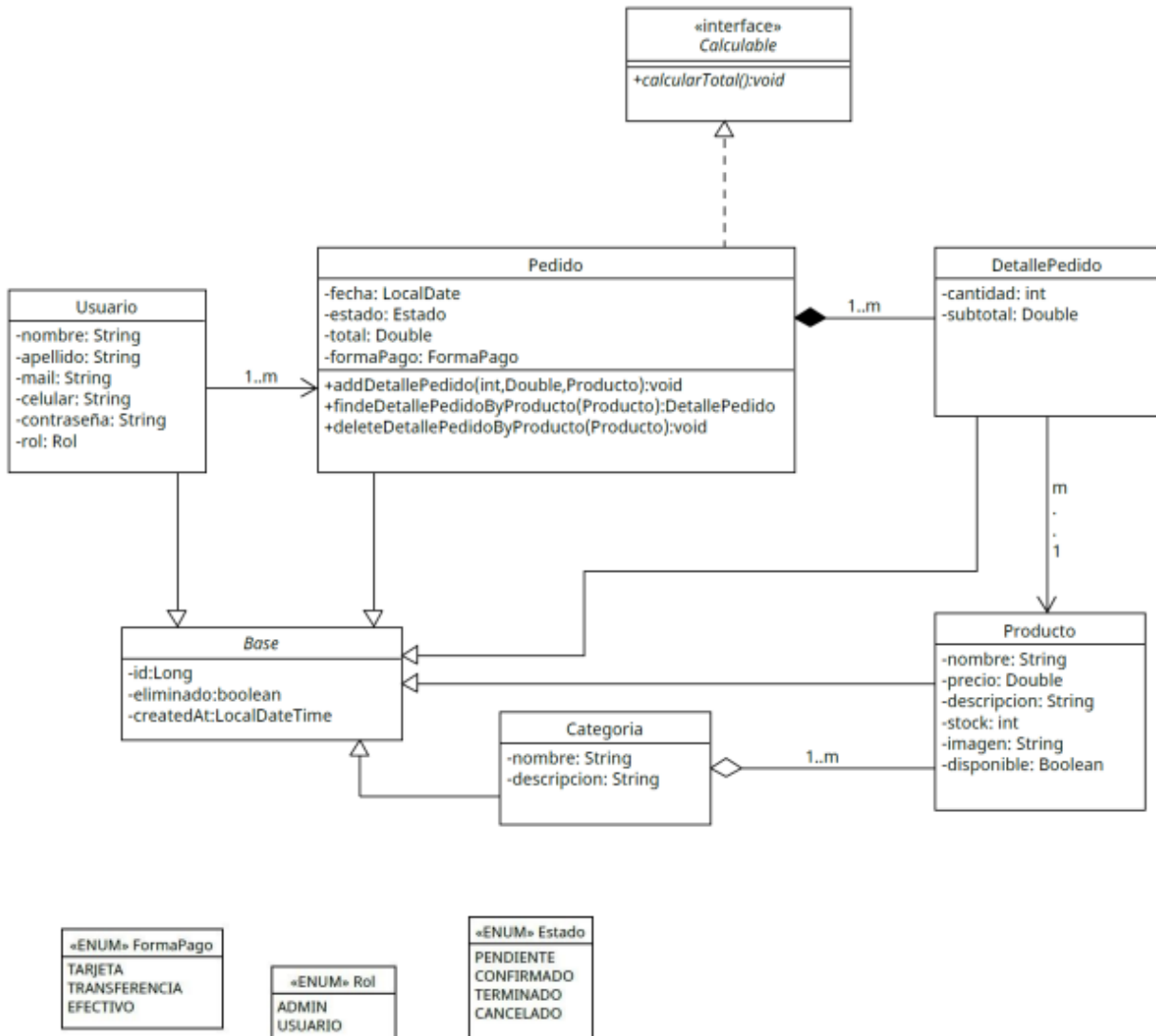
Tecnología	Uso en el proyecto
Java 21	Lenguaje principal del sistema y versión requerida por la consigna.
NetBeans	IDE utilizado para desarrollar, compilar y ejecutar el proyecto.
JDBC	Conexión y operaciones de persistencia con la base de datos.
MySQL	Motor de base de datos relacional.
MySQL Connector/J	Driver JDBC utilizado para conectar Java con MySQL.
Git y GitHub	Control de versiones y trabajo colaborativo por ramas.

4. Modelo y entidades

El sistema se basa en las entidades definidas por el modelo UML de la consigna. Todas las entidades principales heredan de la clase abstracta Base.

- Base: contiene id, eliminado y createdAt.
- Categoria: contiene nombre y descripción. Se relaciona con productos.
- Producto: contiene nombre, precio, descripción, stock, imagen, disponibilidad y categoría.
- Usuario: contiene nombre, apellido, mail, celular, contraseña y rol.
- Pedido: contiene fecha, estado, total, forma de pago, usuario y lista de detalles.
- DetallePedido: contiene cantidad, subtotal y producto asociado.
- Enums: Rol, Estado y FormaPago.
- Interfaz Calculable: define el método calcularTotal(), implementado por Pedido.

DIAGRAMA UML O DE LAS CLASES PRINCIPALES



5. Arquitectura del sistema

El proyecto se organizó en paquetes para mantener una separación clara de responsabilidades.

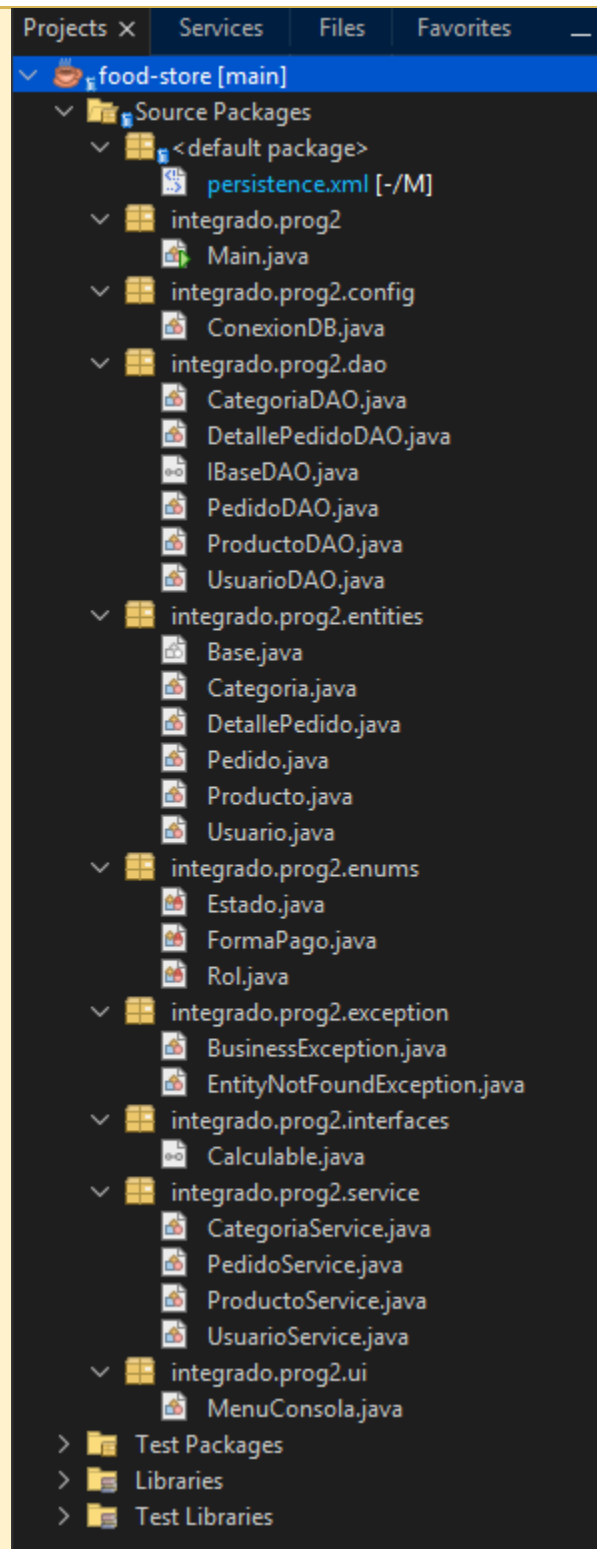
```

src/
├── integrado/prog2/
│   ├── config/
│   ├── dao/
│   ├── entities/
│   ├── enums/
│   ├── exception/
│   ├── interfaces/
│   ├── service/
│   └── ui/
  
```

5.1 Responsabilidades por paquete

Paquete	Responsabilidad
config	Contiene ConexionDB, encargada de leer la configuración y entregar conexiones JDBC.
entities	Contiene las clases del dominio.
dao	Contiene SQL y acceso a datos mediante JDBC.
service	Coordina casos de uso, validaciones y reglas simples de negocio.
ui	Contiene MenuConsola, responsable de la interacción por consola.
exception	Contiene excepciones propias para errores de negocio y entidades no encontradas.
enums	Contiene Rol, Estado y FormaPago.
interfaces	Contiene Calculable.

ESTRUCTURA DEL PROYECTO EN NETBEANS



6. Persistencia y base de datos

La persistencia se implementó con JDBC puro y MySQL. Las operaciones de lectura y escritura se realizan desde los DAO utilizando PreparedStatement, ResultSet y try-with-resources.

6.1 Archivo de configuración

El archivo de configuración de conexión es src/persistence.xml. En este archivo se define el driver, URL, usuario y contraseña de la base de datos. La clase ConexionDB lee este archivo y expone el método getConnection().

```
<persistence>
  <properties>
    <property name="jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
    <property name="jdbc.url"
value="jdbc:mysql://localhost:3306/pedidos_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC"/>
    <property name="jdbc.user" value="fooduser"/>
    <property name="jdbc.password" value=""/>
  </properties>
</persistence>
```

6.2 Tablas principales

- Categoría
- Producto
- Usuario
- Pedido
- DetallePedido

6.3 Transacciones

La creación de pedidos con sus detalles se maneja como una operación transaccional. Primero se inserta el pedido y luego sus detalles. Si ocurre un error al guardar algún detalle, se realiza rollback para evitar inconsistencias.

7. Funcionalidades implementadas

7.1 Categorías

- Listar categorías activas.
- Crear categorías con nombre y descripción.
- Editar categorías existentes.
- Eliminar categorías mediante baja lógica.

7.2 Productos

- Listar productos activos.
- Crear productos asociados a una categoría.
- Validar precio y stock no negativos.
- Editar datos de productos.
- Eliminar productos mediante baja lógica.
- Listar productos por categoría.

7.3 Usuarios

- Listar usuarios activos.
- Crear usuarios.
- Validar mail no vacío y único.
- Editar datos de usuario.
- Eliminar usuarios mediante baja lógica.

7.4 Pedidos y detalles

- Crear pedidos asociados a usuarios.
- Agregar uno o varios detalles por pedido.
- Calcular subtotal por detalle y total del pedido.
- Listar pedidos.
- Ver detalle de pedido.
- Actualizar estado y forma de pago.
- Eliminar pedidos mediante baja lógica.

8. Decisiones técnicas

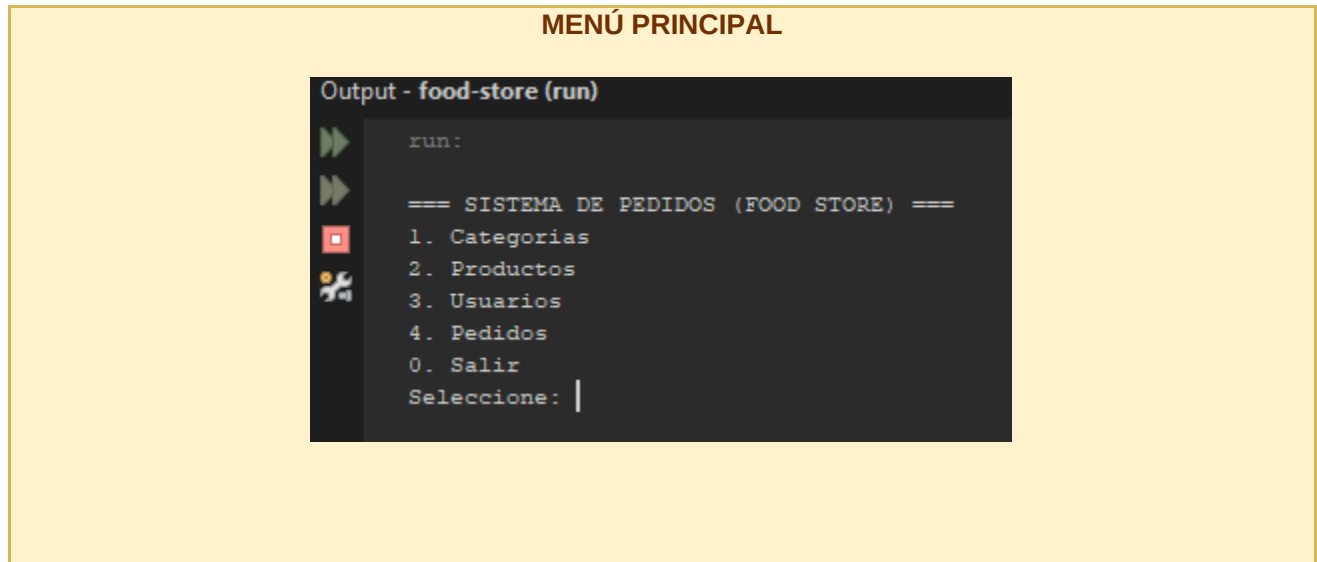
- Se utilizó una aplicación de consola porque la consigna no requiere frontend ni API REST.
- Se aplicó separación por capas para evitar SQL en el menú y Scanner dentro de los DAO.
- Se utilizó una clase Base para compartir atributos comunes.
- Se implementaron enums para valores controlados como rol, estado y forma de pago.
- Se implementó la interfaz Calculable para el cálculo del total del pedido.
- Se utilizó baja lógica para preservar historial y evitar romper relaciones con pedidos o detalles.
- Se centralizó la conexión a base de datos mediante persistence.xml y ConexionDB.

9. Dificultades y soluciones

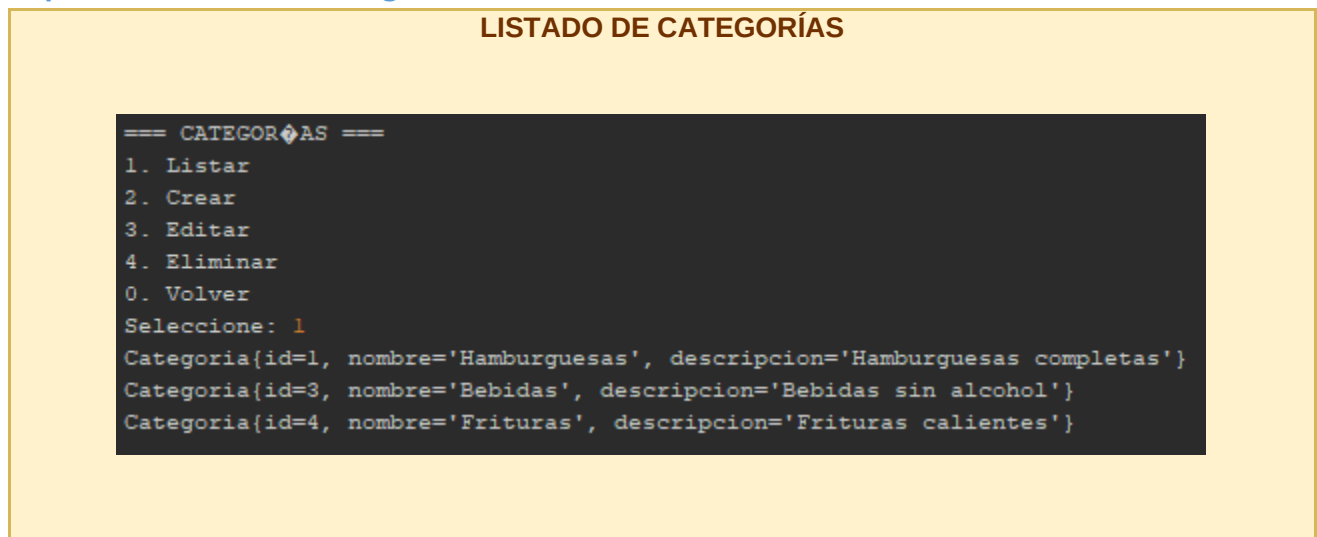
Dificultad	Solución aplicada
Configuración del driver MySQL Connector/J en NetBeans.	Se agregó el archivo mysql-connector-j al Classpath del proyecto.
Error de conexión Public Key Retrieval is not allowed.	Se agregó allowPublicKeyRetrieval=true en la URL JDBC.
Diferencias entre la estructura esperada por el código y la base de datos.	Se revisó y actualizó schema.sql para incluir columnas como fecha en pedido.
Problemas de plataforma Java faltante en NetBeans.	Se configuró la plataforma Java compatible desde Project Properties.
Trabajo en equipo con varias ramas de Git.	Se usaron ramas por módulo y se integraron los cambios en main.

10. Capturas del sistema funcionando

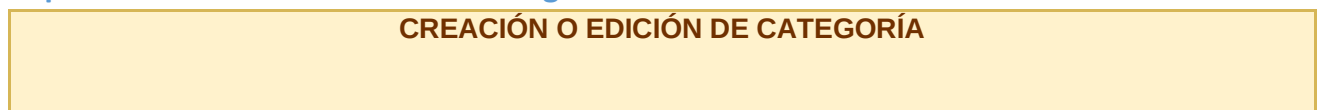
Captura 1: Menú principal



Captura 2: Listado de categorías



Captura 3: Creación o edición de categoría



```

=== CATEGORÍAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Congelados
Descripción: Congelados frescos
Categoría creada con id: 5

```

Captura 4: Listado de productos

LISTADO DE PRODUCTOS

```

=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Listar por categoría
0. Volver
Seleccione: 1
Producto [id=1, nombre=Hamburguesa simple, precio=$3500.0, stock=20, disponible=true, categoria=Hamburguesas]
Producto [id=2, nombre=Pizza muzzarella, precio=$6000.0, stock=15, disponible=true, categoria=Pizzas]
Producto [id=3, nombre=Coca Cola, precio=$1800.0, stock=40, disponible=true, categoria=Bebidas]

```

Captura 5: Listado de usuarios

LISTADO DE USUARIOS

```

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
Usuario{id=1, nombre='Juan', apellido='Pérez', mail='juan@mail.com', celular='1111111111', rol=USUARIO}
Usuario{id=2, nombre='Admin', apellido='Sistema', mail='admin@mail.com', celular='2222222222', rol=ADMIN}

```

Captura 6: Creación de pedido con detalles

CREACIÓN DE PEDIDO CON DETALLES

```

=== PEDIDOS ===
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
5. Ver detalle de pedido
0. Volver
Seleccione: 2
Usuario{id=1, nombre='Juan', apellido='Perez', mail='juan@mail.com', celular='1111111111', rol=USUARIO}
Usuario{id=2, nombre='Admin', apellido='Sistema', mail='admin@mail.com', celular='2222222222', rol=ADMIN}
ID de usuario: 1
Forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione: 1
Producto [id=1, nombre=Hamburguesa simple, precio=$3500.0, stock=20, disponible=true, categoria=Hamburguesas]
Producto [id=2, nombre=Pizza muzzarella, precio=$6000.0, stock=15, disponible=true, categoria=Pizzas]
Producto [id=3, nombre=Coca Cola, precio=$1800.0, stock=40, disponible=true, categoria=Bebidas]
ID de producto: 2
Cantidad: 5
◆Agregar otro producto? S/N: n
Pedido creado con id: 1
Total: $30000.0

```

Captura 7: Listado de pedidos

LISTADO DE PEDIDOS

```

=== PEDIDOS ===
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
5. Ver detalle de pedido
0. Volver
Seleccione: 1
Pedido{id=1, usuario=Juan Perez, fecha=2026-06-22, estado=PENDIENTE, formaPago=TARJETA, total=30000.0}

```

Captura 8: Detalle de pedido

DETALLE DE PEDIDO

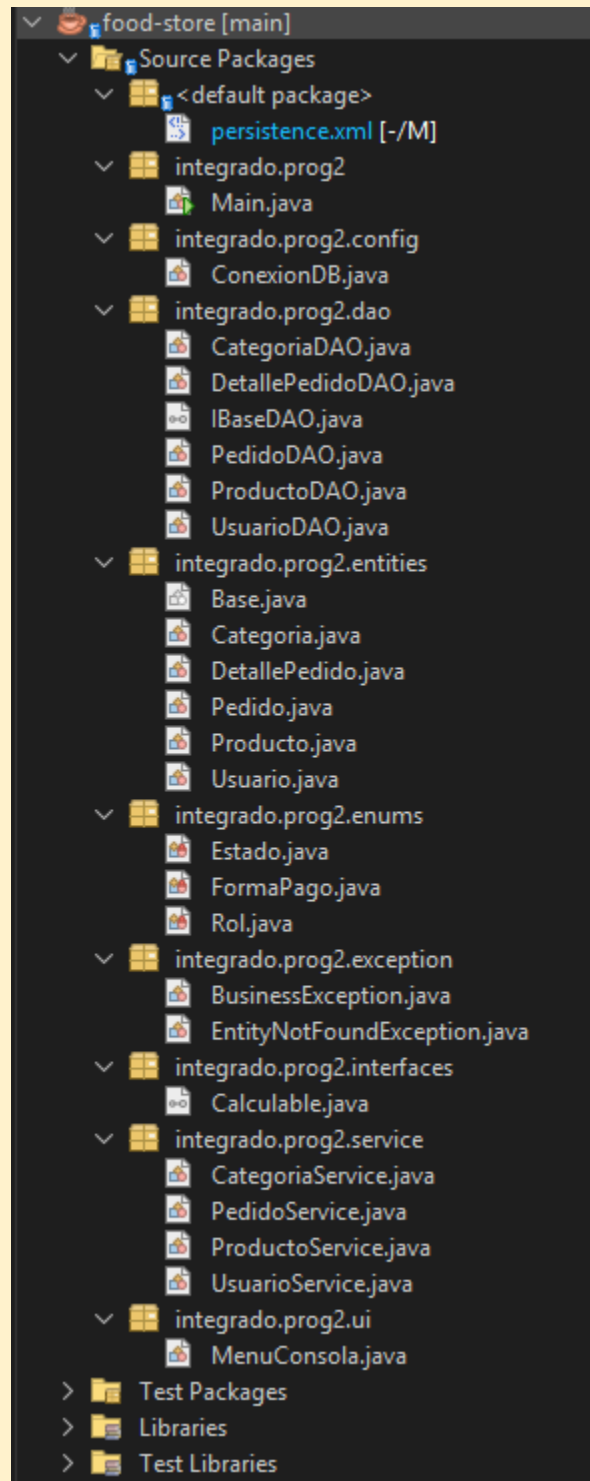
```

=== PEDIDOS ===
1. Listar
2. Crear pedido con detalles
3. Actualizar estado / forma de pago
4. Eliminar
5. Ver detalle de pedido
0. Volver
Seleccione: 5
Pedido{id=1, usuario=Juan Perez, fecha=2026-06-22, estado=PENDIENTE, formaPago=TARJETA, total=30000.0}
ID de pedido: 1
Pedido{id=1, usuario=Juan Perez, fecha=2026-06-22, estado=PENDIENTE, formaPago=TARJETA, total=30000.0}
DetallePedido{id=1, producto=Pizza muzzarella, cantidad=5, subtotal=30000.0}

```

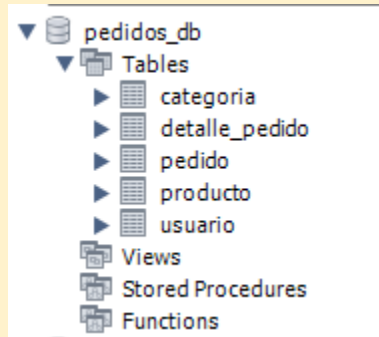
Captura 9: Estructura del proyecto en NetBeans

ESTRUCTURA DEL PROYECTO EN NETBEANS



Captura 10: Tablas en MySQL Workbench

TABLAS EN MYSQL WORKBENCH



11. Pruebas realizadas

- Se verificó que el proyecto compile correctamente.
- Se verificó la conexión a la base de datos MySQL.
- Se listaron categorías, productos y usuarios cargados desde schema.sql.
- Se probó la creación, edición y eliminación lógica de categorías.
- Se probó la creación, edición y eliminación lógica de productos.
- Se probó la creación, edición y eliminación lógica de usuarios.
- Se probó la creación de pedidos con detalles y cálculo de total.
- Se probó el listado de pedidos y la consulta de detalles.
- Se verificó que las bajas lógicas no aparezcan en listados posteriores.

12. Bibliografía / Webgrafía

- Documentación oficial de Java: <https://docs.oracle.com/en/java/>
- Documentación de JDBC: <https://docs.oracle.com/javase/tutorial/jdbc/>
- Documentación oficial de MySQL: <https://dev.mysql.com/doc/>
- MySQL Connector/J: <https://dev.mysql.com/downloads/connector/j/>
- Documentación de Git: <https://git-scm.com/doc>
- Documentación de GitHub: <https://docs.github.com/>

13. Links finales

Repositorio GitHub	https://github.com/genarosenatore/TPI-Prog-2-Castro-Exequiel-Cajales-Benjamin-Piriz-Agustina-Senatore-Genaro.git
Video demostrativo	EL LINK ESTÁ EN LA RAÍZ DEL REPOSITORIO
PDF final	EL PDF ESTÁ EN LA RAÍZ DEL REPOSITORIO